



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Requirements Engineering

Francesco Guerra
francesco.guerra@unimore.it

OBJECTIVES

- ▶ Define requirements engineering.
- ▶ Discuss the steps of requirements engineering.
- ▶ Describe the details of what to do and how to perform the tasks within each step in requirements engineering.
- ▶ Analyze graphical languages that are used during requirements engineering such as data flow diagrams, use cases, and entity-relationship diagram.

Requirement Processing

- ▶ **Requirements** are statements describing user's needs and desires
 - ▶ Requirements must be clearly and fully understood by the software engineers
 - ▶ Requirements should focus on what, not how (they cannot lead to design decisions).
- ▶ One of the top reasons for software project failures is incomplete requirement specifications
- ▶ Managing requirements and user involvement is a key task in software development.

Requirement Engineering

- ▶ **Requirement engineering:** A set of activities related to the development and agreement of the final set of requirement specifications.
- ▶ The following activities are part of the requirements engineering:
 - ▶ Elicitation
 - ▶ Documentation and definition
 - ▶ Specification
 - ▶ Prototyping
 - ▶ Analysis
 - ▶ Review and validation
 - ▶ Agreement and acceptance
- ▶ Not all of these are needed to the same degree for all software projects.
 - ▶ How much and when these activities are practiced and in what sequence is the central theme of the requirement process

Preparing for Requirement Processing



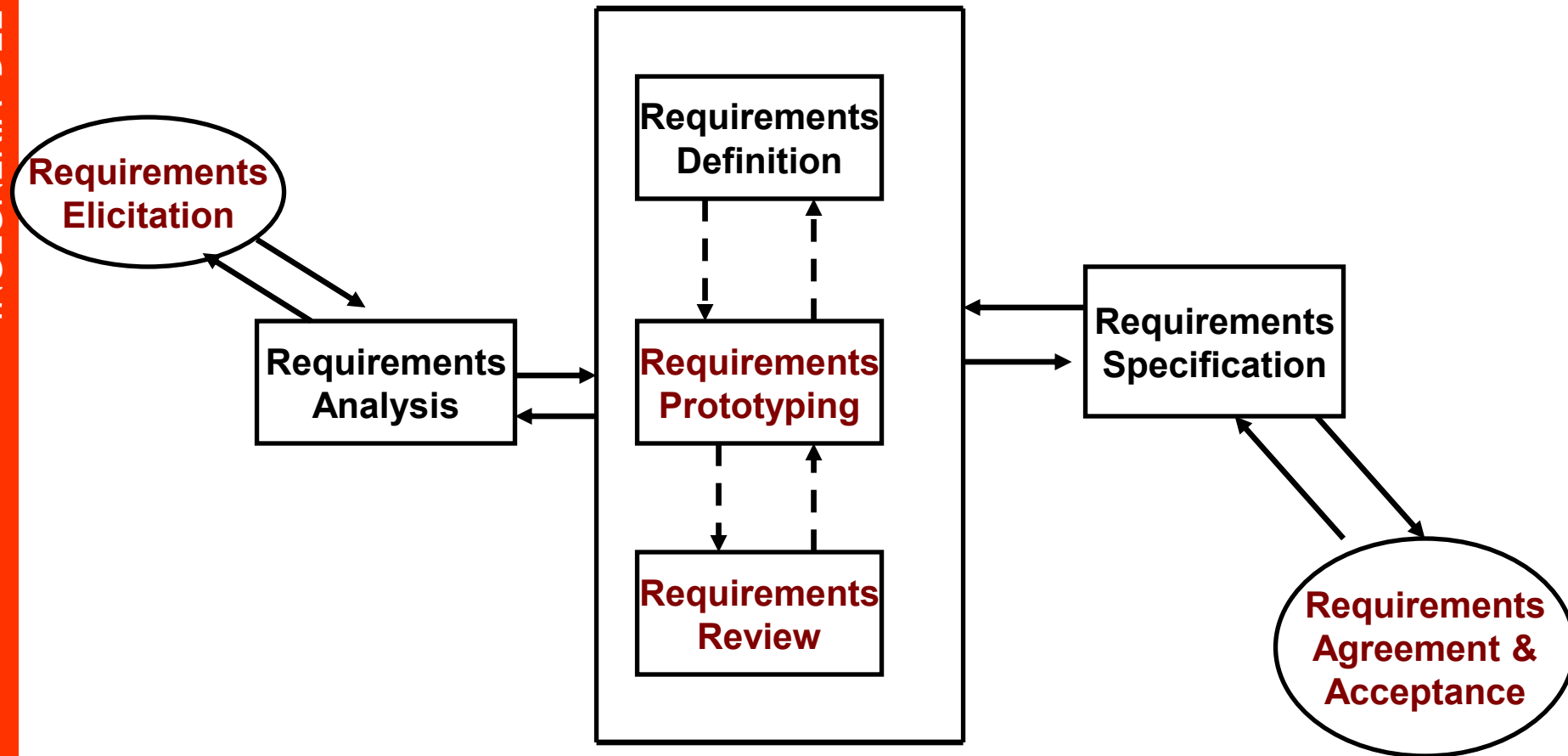
Preparing for Requirement Processing

- ▶ The plan has to be reviewed and agreed upon by all parties involved
 - ▶ Requirements must reflect user needs and involve management, not just the software designer's imagination.
 - ▶ The schedule must be agreed upon by all participants.
 - ▶ Use a flexible schedule to accommodate changes.
- ▶ Acquire all necessary resources (analysts, tools) once the plan is approved.
 - ▶ Ensure skilled and trained personnel, with communication, industry, and technical expertise, are ready for requirements engineering.

Requirements Engineering Process

- ▶ Requirements engineering consists of **several key steps**, from elicitation to specification.
- ▶ Once the requirements specifications are formally agreed upon, they establish **the baseline for the project**.
- ▶ Any change request must be carefully managed through a **change control process** to prevent **uncontrolled** modifications.
- ▶ If changes are not properly controlled, **scope creep** can occur, causing the project to gradually grow without detection.
 - ▶ Requirements scope creep is one of the main causes of schedule delays and cost overruns.

Requirements Engineering Activities



- Red color represents direct user/customer involvement

Requirements Engineering Process (2)

- ▶ The software project implements how the business processes are performed.
 - ▶ Agile software developers have abandoned a segment of requirements engineering.
 - ▶ Constant interactions with users ensure that the requirements are interpreted correctly.
- ▶ The mistake of not taking the time and effort to gather and understand requirements can be costly
- ▶ Negative consequences:
 - ▶ There are no documented requirements to base testing on.
 - ▶ There are no agreed-upon requirements to control scope-creep.
 - ▶ There are no documented requirements to base the customer training or customer support activities on.
 - ▶ It is very difficult to manage project schedule and cost without clear and documented requirements.
- ▶ At the other end of the spectrum are the excessive costs of efforts devoted to prototypes, reviews, the creation of voluminous documents.

Requirements Elicitation and Gathering

- ▶ A requirements analyst must have experience in software development and business domain knowledge.
 - ▶ Communication skills and understanding of the industry domain are crucial for eliciting user requirements.
 - ▶ The analyst must be a good listener and interpreter of user needs.
 - ▶ Familiarity with industry terminology is essential.
- ▶ Users and customers play an active role, continuously clarifying and refining requirements.
- ▶ Two levels of requirements elicitation:
 - ▶ High level: understand the business rationale and justification for the software.
 - ▶ Low level: gather detailed information about users' needs and desires.
- ▶ Analysts should prepare a structured set of questions for users.
 - ▶ Elicitation methods can include: Verbal (interviews, meetings), Written (preformatted forms), Online forms

Requirements Elicitation and Gathering (2)

- ▶ Written and online forms help analysts structure questions and maintain discipline.
 - ▶ Preformatted forms can be too rigid; a verbal follow-up is highly recommended.
- ▶ Direct contact with users often triggers useful follow-up questions and allows users to expand their input.
- ▶ Analysts should gather existing information from:
 - ▶ Business process documents
 - ▶ Business and technical policies
 - ▶ Previous system manuals and other historical information
- ▶ Reading and analyzing documentation is an essential skill for a requirements analyst.

Eliciting High-Level Requirements

- ▶ The requirements analyst must engage with management and executives to understand the business rationale.
- ▶ The business rationale is translated into requirements, expressed as constraints on the software product and project.
- ▶ Key categories of information for the high-level business profile include:
 - ▶ Opportunity / Needs
 - ▶ Justification
 - ▶ Scope
 - ▶ Major Constraints
 - ▶ Major Functionality
 - ▶ Success Factors
 - ▶ User Characteristics

Eliciting High-Level Requirements (2)

- ▶ Opportunity and Needs: focus on business-oriented problems, not technical issues.
 - ▶ Examples:
 - ▶ Excessive inventory expenses
 - ▶ Losing 50% of customer orders due to poor document management
 - ▶ These statements help define the limits and scope of the software project.

Eliciting High-Level Requirements (3)

- ▶ Major Constraints: define limits on the project.
 - ▶ Budget:
 - ▶ Critical for prioritizing detailed requirements.
 - ▶ Helps decide what is needed vs. nice-to-have.
 - ▶ Time Schedule:
 - ▶ Executive needs are often immediate.
 - ▶ Analysts must understand schedule constraints and the actual timeline.

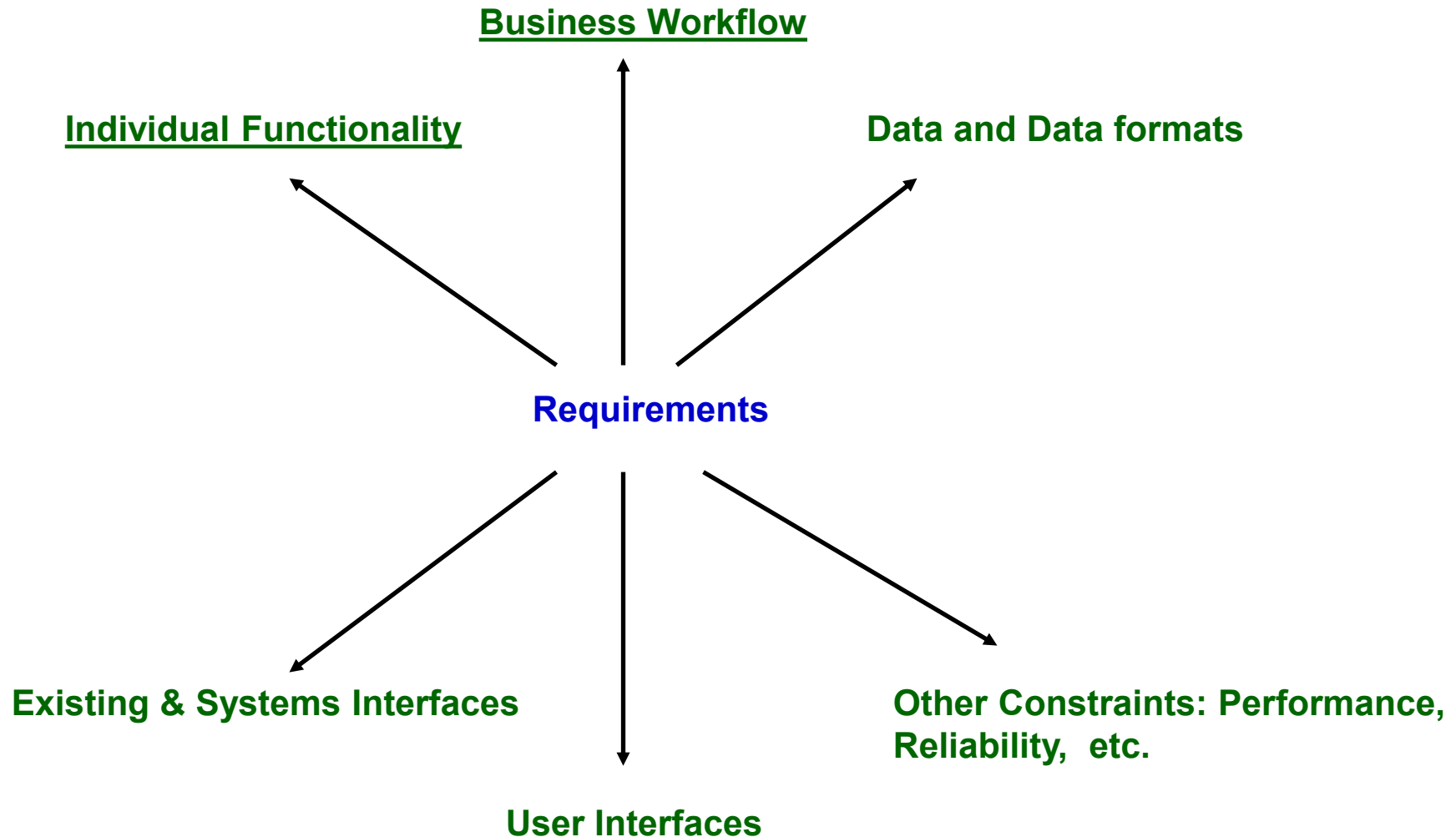
Eliciting High-Level Requirements (4)

- ▶ Major Functionality: define the primary capabilities the new software will deliver.
 - ▶ High-level statements help set customer expectations and guide the scope of the project.
 - ▶ Analysts should translate these into high-level goals and have them reviewed and agreed by all stakeholders.
 - ▶ The project is often considered successful if the high-level business goals are achieved.
- ▶ User Characteristics: understanding the day-to-day users is essential for system adoption and proper training.
 - ▶ Executives may not be day-to-day users, but project success depends on **users** being trained and supported, and the system being well-adapted to their needs

Eliciting Detailed Requirement

- ▶ High-level requirements should be gathered first to define the overall scope.
- ▶ Technically skilled analysts are essential to guide discussions and interpret user input.
- ▶ Users often start talking about possible solutions, especially if a similar system already exists.
- ▶ During these sessions, both analysts and users may explore imaginative and technical ideas.
- ▶ At the detailed level, it is easy to get sidetracked in long discussions, so analysts must maintain focus and control.

The six dimensions of requirements



Eliciting Detailed Requirement

- ▶ Individual functionality is often the starting point of requirements elicitation.
- ▶ Analysts ask users about their problems in terms of functions to be performed.
- ▶ Functionality must be described in the context of:
 - ▶ The business flow
 - ▶ How users perform their tasks in specific scenarios
- ▶ This naturally leads to the development of use cases in object-oriented methodology, where functionalities are linked to actors within a business context.

Eliciting Detailed Requirement

- ▶ Data and data formats must be collected.
 - ▶ At minimum: the application's input and output data.
- ▶ User interfaces: define how input and output are presented (mostly graphical).
 - ▶ Look and flow typically captured by prototyping.
- ▶ Software interfaces: must be clearly specified, covering:
 - ▶ Transfer of control (invocation of the interface)
 - ▶ Transfer of data (direct or via database)
 - ▶ Receipt of responses (success, failure, error types/messages)
 - ▶ Retry capabilities
 - ▶ Errors and messages with corresponding recovery and retry methods

Eliciting Detailed Requirement

- ▶ Reliability, performance, security, and adaptability: key prompts for nonfunctional requirements.
 - ▶ Examples by domain:
 - ▶ Transaction-oriented applications → specify transaction rate (performance).
 - ▶ Life-critical applications → define reliability and availability.
 - ▶ Financial applications → ensure protection of data and data transmission (security).
- ▶ Other constraints may include portability and maintainability.
- ▶ Project-related requirements may mandate use of a specific tool or programming language.

Functional and non functional requirements

- ▶ Functional requirements
 - ▶ the interactions between the system and its environment of implementation (i.e. user and any other external system with which the system interacts).
- ▶ Nonfunctional requirements
 - ▶ aspects of the systems that are not directly related to the functional behavior of the system.

Requirement Analysis

- ▶ After elicitation, requirements are often just an **unorganized set of data**.
- ▶ Requirements analysis involves two main tasks:
 - ▶ **Categorizing / clustering requirements**
 - ▶ **Prioritizing requirements**
- ▶ Categorization can follow different methodologies, usually based on business and usage flow.
- ▶ Key objectives: ensure consistency and completeness.

Requirements Analysis and Clustering by Business Flow

- ▶ A natural clustering of requirements can follow six dimensions:
 1. Individual functionality
 2. Business flow
 3. Data, formats, and information needs
 4. User interfaces
 5. Interfaces with other systems
 6. Constraints such as performance, reliability, and security
- ▶ These categories **are not always mutually exclusive** and may overlap

Requirements Analysis and Clustering by Business Flow (2)

Requirement Area	Prefix	Requirement Statement Numbering
Individual functionality	IF	IF-1.1, IF-1.2, IF-1.3, IF-2.1
Business flow	BF	BF-1, BF-2
Data and data format	DF	DF-1.1, DF-1.2, DF-2.1
User interface	UI	UI-1.1, UI-1.2, UI-2.1
Interface to systems	IS	IS-1
Further constraints	FC	FC-1

- ▶ The prefix identifies the category or the dimension of the requirements.
 - ▶ The numbering scheme of all categories is tied to business flow.
 - ▶ Given a business flow, BF-n, there may not be a requirement in the other five categories related to this business flow.

Requirements Analysis and Prioritization

- ▶ Clustering requirements is part of requirements analysis.
- ▶ Helps identify inconsistencies and possible incompleteness.
- ▶ Requirements may be limited by constraints:
 - ▶ Limited resourcesLimited time
 - ▶ Limited technical capabilities
 - ▶ Prioritization is necessary, especially in multi-release products.
- ▶ Criteria for establishing priority may include:
 - ▶ Current customer demands
 - ▶ Competition and market conditions
 - ▶ Future customer needs
 - ▶ Immediate sales advantage
 - ▶ Critical problems in existing products

Requirements Analysis and Prioritization (2)

Requirement Number	Brief Requirement Description	Requirement Source	Requirement Priority*	Requirement Status
1	One-page query must respond in less than 1 second	A major account marketing representative	Priority 1	Accepted for this release
2	Help text must be field sensitive	Large account users	Priority 2	Postponed for next release

*Priority may be 1, 2, 3, or 4, with 1 being the highest.

- ▶ The analysts perform the prioritization task with the help of other people in the organization.
 - ▶ Sometimes the customers and industry experts are also brought into the discussions.
- ▶ Much of the software requirements prioritization is performed with experienced people and customers using an informal approach
 - ▶ the most persuasive people can bias the priorities of their favorite requirements.
- ▶ Although this approach is not perfect, it is far better than not prioritizing and trying to include everything.

Requirements Analysis and Prioritization (3)

- ▶ A methodical approach is the **Analytical Hierarchy Process - AHP**
 - ▶ It places more rigor into the requirements prioritization process
 - ▶ It is often bypassed by the more marketing-oriented people.
- ▶ Each requirement is compared with each of the other requirements in a pairwise fashion.
 - ▶ An “intensity value” is assigned to this relationship.
 - ▶ The requirement with the highest overall intensity values will essentially be the highest priority requirement.
 - ▶ The requirement with the next highest overall relative intensity value will be the next highest priority requirement and so on.

Analytical Hierarchical Process (AHP)

- Example

	Req 1	Req 2	Req 3
Req 1	1	2	3
Req 2	1/2	1	2
Req 3	1/3	1/2	1

↓ **1.83 3.5 6.0**

	Req 1	Req 2	Req 3
Req 1	.55	.57	.5
Req 2	.27	.29	.33
Req 3	.18	.14	.17

(x,y) → increasing value
of x w.r.t. y (from 1 to 9)

Requirements Prioritization

$$\text{Req 1} = 1.62 / 3 = 54\%$$

$$\text{Req 2} = .89 / 3 = 30\%$$

$$\text{Req 3} = .49 / 3 = 16\%$$

1.62

.89

.49



Example

- ▶ Complex project aiming at creating an app
 - ▶ 8 developers
 - ▶ 33 Functional Requirements
 - ▶ 23 Nonfunctional Requirements
- ▶ Task 1 Identification of the top 10 requirements
 - ▶ Solution
 - ▶ Programmers are asked to evaluate the requirements and voting them
- ▶ Task 2 Establishing Cost and Value
 - ▶ Given by the AHP process
- ▶ Task 3 Ranking the requirements

Example (2)

Requirement ID	Magnus	Elmira	Faegheh	Niclas	Farhan	Sean	Sarah	Total
FR1	9	7	5	6	6	8	6	47
FR2	3	6	4	3	4	5	3	28
FR3	4	6	8	6	7	7	6	44
FR4	6	5	3	6	7	6	6	39
FR5	6	9	3	5	7	6	5	41
FR6	8	10	10	9	9	10	10	66
FR7	10	8	10	10	8	10	8	64
FR8	10	10	10	8	8	10	8	64
FR9	6	8	5	8	9	7	8	51
FR10	3	6	7	5	6	8	4	39
FR11	3	4	3	6	5	5	5	31



Example (3)

► 10 most important requirements

Requirement ID	Title	Requirement Type
FR6	Mobile application - Search	Function
FR7	Mobile application - Search result in a map view	Function
FR8	Mobile application - Search result in a list view	Function
FR24	Restaurant owner manages information	Function
FR27	Administrator verifies restaurant owner	Function
QR6	System response time	Quality
QR7	System Availability	Quality
QR9	System Reliability	Quality
QR12	Communication Security	Quality
QR22	Internet Connection	Quality

Example (4)

► Value

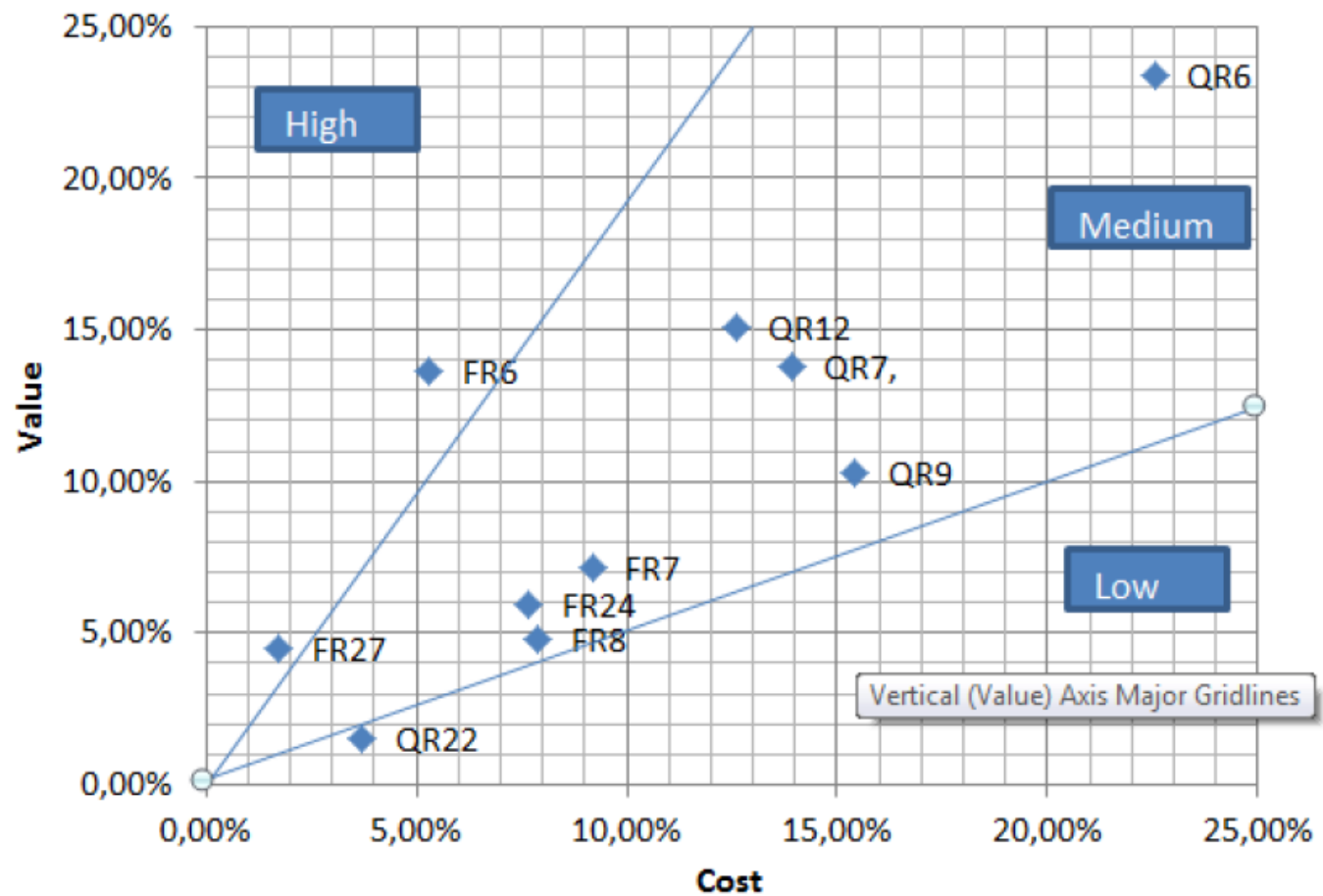
Value	FR6	FR7	FR8	FR24	FR27	QR6	QR7	QR9	QR12	QR22
FR6	1	5	7	7	1/3	1/5	1/3	1/3	5	7
FR7	1/5	1	3	5	6	1/5	1/3	1/3	1/3	5
FR8	1/7	1/3	1	4	5	1/6	1/4	1/4	1/5	3
FR24	1/7	1/5	1/4	1	1/3	1/5	1/5	1/3	5	4
FR27	3	1/6	1/5	3	1	1/9	1/5	1/5	1/7	2
QR6	5	5	6	5	9	1	3	3	2	8

Example (5)

► Cost

COST	FR6	FR7	FR8	FR24	FR27	QR6	QR7	QR9	QR12	QR22
FR6	1	1/5	1/2	3	5	1/7	1/3	1/5	1/3	7
FR7	5	1	1/5	7	3	1/5	1/3	1/5	3	5
FR8	2	5	1	3	5	1/9	1/5	1/6	1/5	7
FR24	1/3	1/7	1/3	1	3	1/3	2	1/5	3	7
FR27	1/5	1/3	1/5	1/3	1	1/7	1/6	1/7	1/6	2
QR6	7	5	9	3	7	1	3	2	3	9
QR7	3	3	5	1/2	6	1/3	1	2	3	7
QR9	5	5	6	5	7	1/2	1/2	1	3	5

Example (6)



	FR6	FR7	FR8	FR24	FR27	QR6	QR7	QR9	QR12	QR22
Cost	5,31%	9,18%	7,91%	7,65%	1,73%	22,55%	13,92%	15,43%	12,60%	3,71%
Value	13,60%	7,12%	4,80%	5,93%	4,45%	23,41%	13,78%	10,31%	15,06%	1,53%

Example (7)

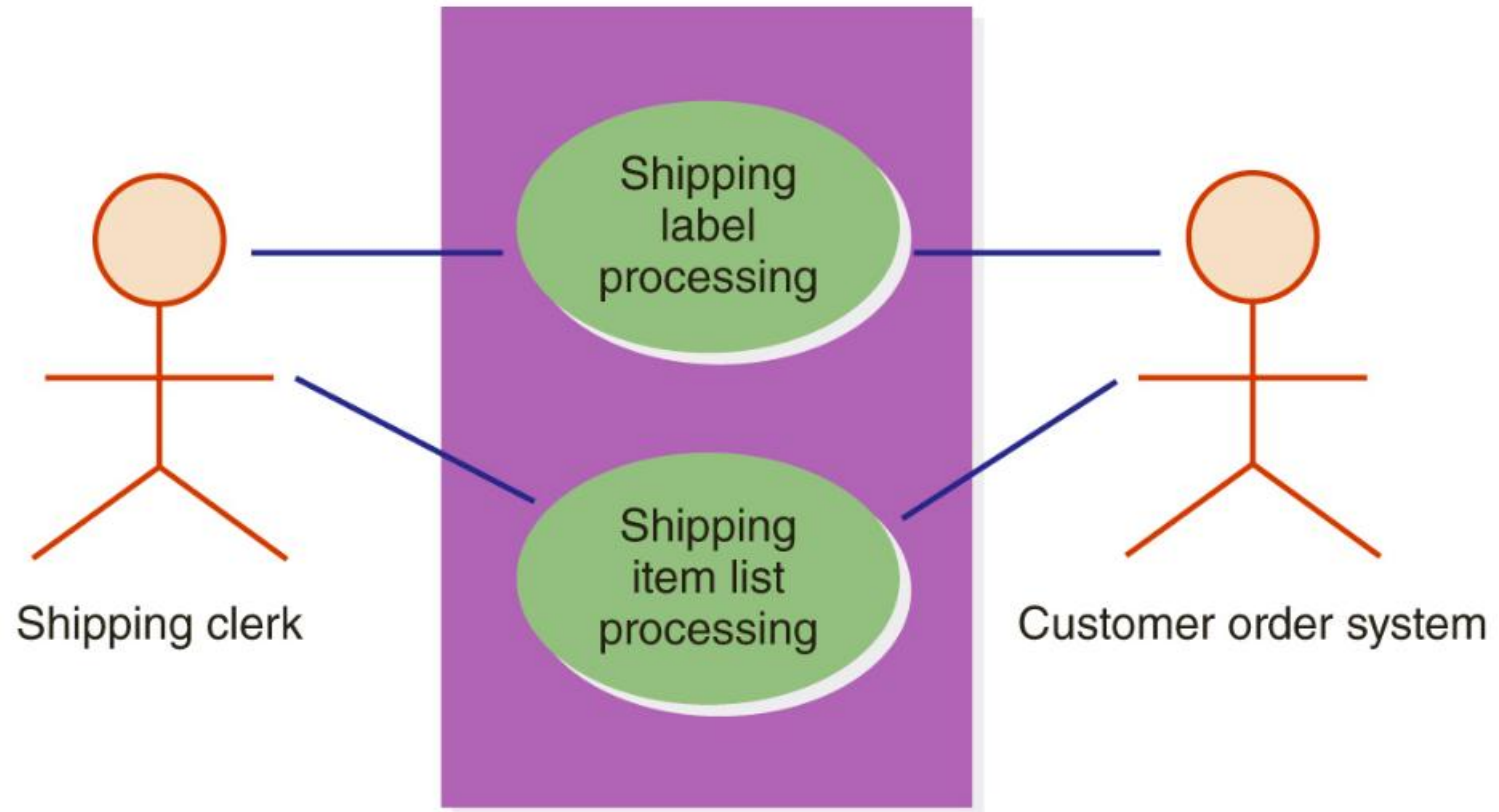
► Alternative approach

Requirement ID		Priority by Stakeholder			
		User	Restaurant owner	Administrator	Customer
FR1	Magnus	1	-2	-2	2
	Elmira	1	-1	-2	2
	Niclas	1	-2	-2	1
	Faegheh	1	-2	-2	1
	Sarah	1	-1	-1	1
	Average	1	-1,6	-1,8	1,4
	Sum	-1			
	Rank of Req.	13			
FR2	Magnus	1	-2	-2	1
	Elmira	1	-1	-2	1
	Niclas	0	-2	-2	1
	Faegheh	-1	-2	-2	0
	Sarah	0	-2	-2	0
	Average	0,2	-1,8	-2	0,6
	Sum	-3			
	Rank of Req.	34			
		User	Restaurant owner	Administrator	Customer

Requirements Analysis and Clustering with Object-Oriented Use Cases

- ▶ Object-oriented use cases help describe and analyze system requirements.
- ▶ They also support design and testing activities.
- ▶ A use case typically includes:
 - ▶ Basic functionality
 - ▶ Scenario describing the functionality
 - ▶ Preconditions and postconditions
 - ▶ Error conditions and alternative flows
- ▶ In OO approaches, elicitation and analysis are often intertwined during use case development

Use case diagram



Requirements Analysis

- ▶ Once the information is collected, the system boundaries are identified.
 - ▶ This is the product that is to be developed.
 - ▶ The identification of system boundaries starts with the delineation of what is included and excluded from the system but may still be needed for interfacing with the system.
- ▶ Performing system boundary analysis as a part of the requirements analysis can further improve and firm up the completeness of the requirements.
- ▶ In OO use-case methodology, the steps of identifying the following factors serve as a good requirements analysis methodology:
 - ▶ Actors
 - ▶ Related use cases
 - ▶ Boundary conditions

Alternative: FURPS+ model

- ▶ Functionality: the main product features
- ▶ Usability: the ease with which the user can learn to operate, prepare inputs for and interpret outputs of a system
- ▶ Reliability: the ability of a system to perform its required functions under stated conditions for a specific period of time
- ▶ Performance: quantifiable attributes as response time, accuracy, ...
- ▶ Supportability: easy of changes to the system after deployment (adaptability, maintainability, portability ...)
- ▶ +
 - ▶ Design requirements
 - ▶ Implementation requirements
 - ▶ Interface requirements
 - ▶ Physical requirements

Requirements Analysis and Clustering by Viewpoint-Oriented Requirements Definition

- ▶ Viewpoint-Oriented Requirements Definition (VORD): a methodology for **requirements elicitation and analysis**.
- ▶ Based on the idea that **different stakeholders perceive requirements differently**:
 - ▶ **Customers**: focus on business goals and outcomes.
 - ▶ **Daily users**: focus on interaction and usability.
 - ▶ Large systems → multiple users with diverse priorities and emphases.
- ▶ Implications:
 - ▶ The **same requirement** may appear in different forms or contexts.
 - ▶ Requirements may **overlap** and need to be reorganized or combined.
- ▶ VORD methodology steps:
 - ▶ Identify viewpoints and stakeholders.
 - ▶ Document the viewpoints.
 - ▶ Map viewpoints to the system and its services.

Requirements Traceability

- ▶ Requirements traceability: the ability to track each requirement through development, testing, packaging, and delivery.
- ▶ No functionality or property should remain unaccountable.
- ▶ Types of traceability (Kotonya & Sommerville, 1998):
 - ▶ Backward from traceability: links requirement to its source or creator.
 - ▶ Forward from traceability: links requirement to design and implementation.
 - ▶ Backward to traceability: links design/implementation back to requirements.
 - ▶ Forward to traceability: links preceding documents to the requirements.
- ▶ Requirements may have related dependencies (co-requirements, pre/post relationships).
- ▶ Requirements relationship matrices can be developed.
- ▶ Unique identification ensures each requirement is traceable.

Requirements Definition, Prototyping, and Reviews

- ▶ Requirements definition, prototyping, and review are represented as three separate activities
 - ▶ In practice, they actually overlap
 - ▶ They should be looked upon as a set of iterative activities within the broader context of analysis.
- ▶ Requirements definition involves formally spelling out the requirements.
 - ▶ The notation used is a language accompanied with other notations.
 - ▶ One of the simplest notations for defining requirements in terms of natural language is the input-process-output

Requirement Number	Input	Process	Output
12: Customer order	• Items by type and quantity • Submit request	• Accept the items and respective quantities	• Display acceptance message • Ask for confirmation message

Requirements Definition, Prototyping, and Reviews (2)

- ▶ Popular notations supporting the process are
 - ▶ UML and in particular Use cases, Activity Diagram and DFD (Data Flow Diagram)
 - ▶ ER

Requirements Definition, Prototyping, and Reviews (3)

- ▶ As part of the requirements gathering and analysis, the user interface is a vital component.
- ▶ Two main aspects of user interface must be analyzed during requirements time:
 - ▶ Visual looks and display
 - ▶ Interaction with people and flow
- ▶ In the early days, the user interfaces were modeled with paper boards
 - ▶ low-fidelity prototyping.
- ▶ Today, the user interface is prototyped with machine-executable code during requirements gathering and analysis time.
- ▶ These prototypes are sometimes kept and turned into the final release code.

Requirements Definition, Prototyping, and Reviews (4)

- ▶ Reviewing requirements with users and customers is essential in requirements analysis and prototyping.
 - ▶ A single requirement error can propagate into multiple design and programming errors.
- ▶ Reviews can be:
 - ▶ Informal and frequent (as in Agile methods)
 - ▶ Formal inspections (IBM, 1970s)
 - ▶ Often a hybrid approach is used by practitioners.
- ▶ After reviews, modifications and corrections must be applied.
- ▶ Changes should be clearly documented to prevent errors from propagating to design, coding, testing, and final release.

Requirements Specification and Requirements Agreement

- ▶ After requirements are analyzed and reviewed, they should be documented in a requirements specification document.
- ▶ The level of detail depends on:
 - ▶ Project size and complexity
 - ▶ Planned multiple follow-on releases
 - ▶ Expected customer support activities
 - ▶ Knowledge and experience of developers in the domain
- ▶ A detailed specification is essential for a large user base to enable organized maintenance and support.
- ▶ If developers, testers, and designers have limited domain knowledge, a detailed specification is essential.

Systems and software engineering – Life cycle processes – Requirements engineering

- ▶ This document
 - ▶ specifies the required processes implemented in the engineering activities that result in requirements for systems and software products;
 - ▶ provides guidelines for applying the requirements and requirements-related processes;
 - ▶ specifies the required information items produced through the implementation of the requirements processes;
 - ▶ specifies the required contents of the required information items;
 - ▶ provides guidelines for the format of the required and related information items.
- ▶ The Requirement process shall produce the following information items
 - ▶ Business requirement specification (BRS);
 - ▶ Stakeholder requirements specification (StRS);
 - ▶ System requirements specification (SyRS);
 - ▶ Software requirements specification (SRS).

ISO/IEC/IEEE 29148

- ▶ **Business requirements specification (BRS) content**
 - ▶ Business purpose
 - ▶ Business scope
 - ▶ Business overview
 - ▶ Major Stakeholders
 - ▶ Business environment
 - ▶ Mission, goals and objectives
 - ▶ Business model
 - ▶ Information environment
 - ▶ Business processes

ISO/IEC/IEEE 29148

- ▶ **Business requirements specification (BRS) content**
 - ▶ Business operational policies and rules
 - ▶ Business operational constraints
 - ▶ Business operational modes
 - ▶ Business operational quality
 - ▶ Business structure
 - ▶ High-level operational concept
 - ▶ High-level operational scenarios
 - ▶ Other high-level life-cycle concepts
 - ▶ Project constraints

ISO/IEC/IEEE 29148

- ▶ **Stakeholder requirements specification (StRS) content**
 - ▶ Stakeholder purpose
 - ▶ Stakeholder scope
 - ▶ Overview
 - ▶ Stakeholders
 - ▶ Business environment
 - ▶ Mission, goals and objectives
 - ▶ Business model
 - ▶ Information environment

ISO/IEC/IEEE 29148

- ▶ **Stakeholder requirements specification (StRS) content**
 - ▶ System processes
 - ▶ System operational policies and rules
 - ▶ Operational constraints
 - ▶ System operational modes and states
 - ▶ System operational quality
 - ▶ User requirements
 - ▶ Operational concept
 - ▶ Operational scenarios
 - ▶ Other detailed concepts of proposed system
 - ▶ Project constraints

ISO/IEC/IEEE 29148

- ▶ **System requirements specification (SyRS) content**
 - ▶ System purpose
 - ▶ System scope
 - ▶ System overview
 - ▶ Functional requirements
 - ▶ Usability requirements
 - ▶ Performance requirements
 - ▶ System interface requirements

ISO/IEC/IEEE 29148

- ▶ **System requirements specification (SyRS) content**
 - ▶ System modes and states
 - ▶ Physical characteristics
 - ▶ Environmental conditions
 - ▶ System security requirements
 - ▶ Information management requirements
 - ▶ Policy and regulation requirements
 - ▶ System life cycle sustainment requirements
 - ▶ Packaging, handling, shipping and transportation requirements
 - ▶ Verification
 - ▶ Assumptions and dependencies

ISO/IEC/IEEE 29148

- ▶ **Software requirements specification (SRS) content**
 - ▶ Purpose
 - ▶ Scope
 - ▶ Product perspective
 - ▶ Product functions
 - ▶ Limitations
 - ▶ Assumptions and dependencies
 - ▶ Apportioning of requirements

ISO/IEC/IEEE 29148

- ▶ **Software requirements specification (SRS) content**
 - ▶ **Specified requirements**
 - ▶ describe every input (stimulus) into the software system, every output (response) from the software system, and all functions performed by the software system in response to an input or in support of an output.
 - ▶ **External interfaces**
 - ▶ **Usability requirements**
 - ▶ **Performance requirements**
 - ▶ **Logical database requirements**
 - ▶ **Standards compliance**
 - ▶ **Software system attributes**

How to describe requirements

Suggestion

Introduction	Actors
	Overview of the functionality
Input	Generic description
Process	Data Validation
	Sequence process
	Handling Errors and Messages
	Parameters with some effects on the output
Output	

How to describe requirements

Suggestion

Code	Reference Area	Name
Input		
Process		
Output		



References

- ▶ Frank Tsui, Orlando Karam, Barbara Bernal, Software engineering (Fifth Edition), Jones & Bartlett Learning
- ▶ ISO 29148:2018

MATERIALE DIDATTICO INTERNO DEL CORSO DI Ingegneria del Software.

La diffusione e pubblicazione on line del materiale è assolutamente proibita.